

download_then_webready_challenge.sh

Companion user guide (§11.4.18) for challenges/scripts/download_then_webready_challenge.sh.

Overview

This is the anti-bluff Challenge for the **dual-version media pipeline** (README “Dual-version media pipeline”). It goes one step further than `download_completes_challenge.sh` (the constitution §11.4.108 ARTIFACT rule): after the original download reaches finished via the MeTube /add API, it **stat-verifies BOTH the original file AND the media_postprocessor derivative**, then `ffprobe`-asserts the derivative is genuinely playable.

- In video mode it proves a `webready-<base>.mp4` appeared next to the original, with **H.264 video + AAC audio + non-zero duration + faststart** (moov atom before mdat).
- In audio mode it proves a `<base>.mp3` appeared, with an **mp3 stream + non-zero duration**.

The guiding rule (from the script header): *“if I deleted the media_postprocessor implementation, would this test still pass?”* — the answer must be **NO**. The `webready/ffprobe` assertions FAIL when the artifact is absent, so a green run is real proof the feature works for the end user, not merely that `/add` returned 200.

Prerequisites

- A **running stack** in the `no-vpn` profile: MeTube (`metube-direct`) **and** the `media_postprocessor` sidecar must both be up (`make init` then `./start_no_vpn`).
- `DOWNLOAD_DIR` must exist on the host and be the same directory MeTube and the `postprocessor` write into (the script reads files from the **host’s** point of view, not from inside a container).
- Host tools on PATH: `curl`, `python3`, `ffprobe`, `stat`.
- The script sources `.env` (when present) for `DOWNLOAD_DIR` and `METUBE_DIRECT_PORT`, and `lib/container-runtime.sh` for runtime detection (parity with peer challenges).

Usage

```
# Video (default): assert webready-<base>.mp4 with h264 + aac + faststart
bash challenges/scripts/download_then_webready_challenge.sh
```

```
# Audio: assert <base>.mp3
MODE=audio bash challenges/scripts/
    download_then_webready_challenge.sh
```

Environment overrides (all optional):

Variable	Default	Meaning
DOWNLOAD_DIR	from .env, else /mnt/ downloads	Host library directory to scan
API_URL	http://localhost:\$ {METUBE_DIRECT_PORT:-8088}	MeTube Direct base URL
MODE	video	video or audio — which artifact to assert
TEST_URL	a Vimeo test clip	Override the test media URL
TIMEOUT	180	Seconds to wait for / history status=finished
WEBREADY_TIMEOUT	120	Seconds to wait for the postprocessor artifact

Exit codes: 0 = PASS (every artifact proven playable), 1 = FAIL.

Edge cases

- **/add not accepted** — if /add does not return status:ok the run FAILs immediately (asserts on the JSON body, not on the HTTP code — see CONST-034).
- **Download errors / times out** — a status=error history entry FAILs with the MeTube msg; not reaching finished within TIMEOUT FAILs.
- **Original missing or < 1KB** — the before/after find snapshot diff must surface a new, non-webready, non-transient (*.part/*.ytdl/*.tmp) file larger than 1 KB, or the run FAILs.
- **Derivative never appears** — if the webready-*.mp4 / .mp3 does not show up within WEBREADY_TIMEOUT, the run FAILs. This FAIL is the anti-bluff signal: the postprocessor did not produce the artifact.
- **Degenerate derivative** — an artifact smaller than 1 KB, or with the wrong codec / a <= 0.5s duration / (video) moov after mdat, FAILs the ffmpeg assertions.
- **Cleanup** — on success the test download is removed from MeTube history; the before/after temp files are removed via an EXIT trap.

Internal behaviour

Five-step pipeline (set `-e`; colourised PASS/FAIL output):

1. **[1/5] Submit** the test URL via `POST /add`; require `status:ok` in the body.
2. **[2/5] Wait** up to `TIMEOUT` for the URL to reach `status=finished` in `/history` (parsing queue/pending/done with `python3`).
3. **[3/5] Verify the original** — diff a before/after find `-printf '%s %p'` snapshot, pick the new non-webready/non-transient file as the original, assert it is `>= 1KB`, derive `<base>` (basename without extension).
4. **[4/5] Wait** up to `WEBREADY_TIMEOUT` for the postprocessor artifact next to the original: `webready-<base>.mp4` (video) or `<base>.mp3` (audio); assert it exists and is `>= 1KB`.
5. **[5/5] ffmpeg-probe-assert** the artifact is genuinely playable:
 - **video:** `v:0 codec_name == h264, a:0 codec_name == aac, format.duration > 0.5s`, **and faststart** — an inline `python3` atom walker reads the top-level MP4 atom order and requires `moov` before `mdat` (the same check `transcoder.py::has_faststart` performs).
 - **audio:** `a:0 codec_name == mp3 and format.duration > 0.5s`.

These assertions mirror the validator the pipeline itself runs (`media_postprocessor/transcoder.py::_validate_webready / _validate_mp3`), so a PASS confirms the produced derivative meets the same contract the service enforces.

Related scripts

- `challenges/scripts/download_completes_challenge.sh` — the ARTIFACT-rule reference template this Challenge extends (original-file proof only).
- `challenges/scripts/run_all_challenges.sh` — runs the full Challenge bank.
- `media_postprocessor/transcoder.py` — the derivation + `ffmpeg`-validation engine whose output this Challenge proves.

Last verified

2026-06-15 — behaviour documented against the in-tree script and the `media_postprocessor` implementation it exercises.